

Формальные грамматики, осень 2023 г.
Лекция 1: Четыре определения обыкновенных грамматик,
их равносильность. Нормальный вид Грейбах,
преобразование грамматики к нему (начало)*

Александр Охотин

7 сентября 2023 г.

Содержание

1	Введение	1
2	Определения грамматик	2
2.1	Определение через дерево разбора	4
2.2	Определение через логический вывод	4
2.3	Определение через перезапись строк	5
2.4	Определение через языковые уравнения	5
2.5	Равносильность четырёх определений	9
3	Нормальные виды грамматик	11
3.1	Пустые правила	11
3.2	Единичные правила	11
3.3	Нормальный вид Хомского	11
3.4	Левая рекурсия	12
3.5	Нормальный вид Грейбах	13

1 Введение

Бывают языки с простым синтаксисом, для некоторых языков требуются сложные определения — в каждом случае нужна модель синтаксиса с подходящей выразительной мощностью. Для этого находятся и изучаются различные семейства формальных грамматик. Для каждого нужно найти определение и понять его выразительную мощность — а именно, какие языки в нём задаются, а какие нет.

Общая черта всех (осмысленных) видов формальных грамматик в том, что в грамматике задаётся некоторый набор свойств (синтаксических категорий, «нетерминальных символов»), и каждый фрагмент предложения может обладать или не обладать каждым из этих свойств. Грамматика назначает некоторым фрагментам предложения — «составляющим» — эти свойства, что проделывается от меньших составляющих к большим. В большинстве видов грамматик, составляющие — это **подстроки** предложения.

*Краткое содержание лекций, прочитанных студентам факультета МКН СПбГУ в осеннем семестре 2023–2024 учебного года. Страница курса: http://users.math-cs.spbu.ru/~okhotin/teaching/fg_2023/.

В правилах грамматики показывается, как можно соединить меньшие составляющие с определённым смыслом в составляющие большего размера с определённым смыслом. Если составляющие — подстроки, то они обычно соединяются операцией **конкатенации**. Например, приписыванием подлежащего с зависимыми словами “**Всякий человек**” к сказуемому “**смертен**” получается грамотное русское предложение. Аналогично, последовательно записывая ключевое слово “**if**” к выражению “ $x < 0$ ”, к ещё одному ключевому слову “**then**” и к оператору $x = 0$; , можно получить правильный условный оператор на некотором языке программирования. В большинстве видов формальных грамматик, конкатенация — это единственный способ получить новые строки из существующих.

Наконец, правила грамматики определяют, какие составляющие, в каких сочетаниях и в каком порядке можно соединять. Для этого обычно используется логическая связка — **дизъюнкция** — задающая *выбор* между несколькими возможными синтаксическими конструкциями. Например, оператор в языке программирования — это оператор присваивания, *или* условный оператор, *или* оператор цикла, и т.д.

Основная модель грамматик — *обыкновенные грамматики* (по старинке называемые «бесконтекстными»¹). В этих грамматиках составляющие — подстроки, конкатенация — единственная операция над составляющими, дизъюнкция — единственная логическая операция. Это общепринятая модель синтаксиса — наиболее важное семейство формальных грамматик. Все остальные семейства определяются относительно них: есть более слабые модели, с ограничениями на использование конкатенации и дизъюнкции; есть и более сильные модели, допускающие иные операции, кроме конкатенации и дизъюнкции, и даже более сложные составляющие, чем подстроки.

Для каждого семейства грамматик есть два основных вопроса. Во-первых, какие синтаксические конструкции можно описать такими грамматиками, и какие нельзя? (выразительная мощность) Во-вторых, что можно сделать с такими грамматиками на компьютерах, и насколько эффективно? (алгоритмы и сложность) Хорошая модель позволяет задать достаточно, и одновременно с ней не очень трудно работать; критерии в каждом приложении свои, и потому полезно искать и изучать различные модели с удачными наборами свойств.

В качестве точки отсчёта принимаются обыкновенные грамматики. Они позволяют задать много естественных синтаксических конструкций, а что-то, наоборот, задать не позволяют. При этом их можно разобрать за время n^ω , а также в классе NC^2 . У них есть несколько важных частных случаев, а также известно несколько осмысленных обобщений.

2 Определения грамматик

Определение 1. (Формальная) грамматика — это четвёрка $G = (\Sigma, N, R, S)$, состоящая из следующих компонентов.

- Конечное множество **символов** Σ — алфавит определяемого языка.
- Конечное множество N — это множество определяемых в грамматике свойств строк, которым всякая строка над алфавитом Σ обладает или не обладает. Обозначаются обычно буквами $A, B, C, \dots$

В информатике элементы N традиционно называют «нетерминальными символами» («нетерминалами») — отсюда буква N — поскольку пути в дереве разбора на

¹Потому что когда-то очень давно Ноам Хомский изучал возможности определения контекстно-зависимых правил — из этих попыток ничего не вышло, а нелепое название осталось. Оно, к сожалению, используется повсеместно, от этого не уйти.

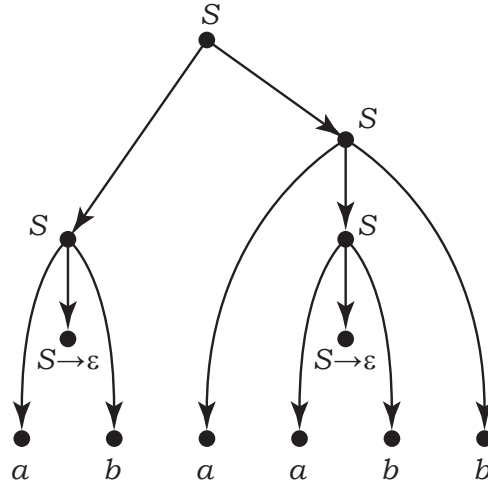


Рис. 1: Дерево разбора строки $abaabb$ по грамматике из примера 1.

них не заканчиваются. В лингвистике они называются синтаксическими категориями.

- Конечное множество R правил грамматики, каждое из которых описывает возможную структуру строк со свойством $A \in N$ в виде конкатенации $u_0 B_1 u_1 \dots B_\ell u_\ell$, где $B_1, \dots, B_\ell$ ($\ell \geq 0$) — все нетерминальные символы, на которые ссылается правило, а любые символы, написанные между ними, образуют строки $u_0, u_1, \dots, u_\ell$.

$$A \rightarrow u_0 B_1 u_1 \dots B_\ell u_\ell \quad (A \in N, \ell \geq 0, B_1, \dots, B_\ell \in N, u_0, u_1, \dots, u_\ell \in \Sigma^*)$$

- Начальный символ $S \in N$, от «sentence», обозначает множество всех синтаксически правильных строк, определяемых в грамматике

Общий вид правил часто записывается как $A \rightarrow X_1 \dots X_\ell$, где $X_1, \dots, X_\ell \in \Sigma \cup N$ ($\ell \geq 0$) — все символы и нетерминальные символы, конкатенация которых записана в правиле. Дальнейшее обозначение: $A \rightarrow \alpha$, где $\alpha = X_1 \dots X_\ell$ — строка над алфавитом $\Sigma \cup N$.

Несколько правил $A \rightarrow \alpha_1, \dots, A \rightarrow \alpha_m$ для одного и того же нетерминального символа $A \in N$ записывают возможную структуру строк с этим свойством. Это такой же оператор выбора, как и в регулярных выражениях, и потому такие правила обычно записываются следующим образом.

$$A \rightarrow \alpha_1 \mid \dots \mid \alpha_m$$

Пример 1. Грамматика для языка Дика записывается как $G = (\Sigma, N, R, S)$, где $\Sigma = \{a, b\}$, $N = \{S\}$ и $R = \{S \rightarrow aSb, S \rightarrow SS, S \rightarrow \varepsilon\}$. Такую грамматику записывают следующим образом.

$$S \rightarrow aSb \mid SS \mid \varepsilon$$

Дерево разбора строки $w = abaabb$ в соответствии с этой грамматикой приведено на рис. 1.

2.1 Определение через дерево разбора

Дерево разбора — это уже само по себе формальное определение.

Дерево разбора строки $w \in \Sigma^*$. Потомки каждой внутренней вершины упорядочены, откуда вытекает порядок на листьях. Листья слева направо помечены символами строки w . Каждая внутренняя вершина помечена некоторым нетерминальным символом $A \in N$, и если $X_1, \dots, X_\ell$ — её потомки, то в грамматике должно быть правило $A \rightarrow X_1 \dots X_\ell$. Для правила $A \rightarrow \varepsilon$ вершина помечена символом A , не имеет потомков, но всё равно рассматривается в качестве внутренней.

Язык, задаваемый грамматикой, $L(G) \subseteq \Sigma^*$, содержит все строки, для которых есть дерево разбора.

2.2 Определение через логический вывод

Логическая система, в которой выводятся все верные высказывания вида “строка w имеет свойство A ”, обозначаемые через $A(w)$.

Пример 1(D). В грамматике из примера 1 определён один нетерминальный символ S , соответствующий понятию правильно вложенной строки. Поэтому высказывание вида $S(w)$ можно прочесть как “ w — правильно вложенная строка”.

Утверждение о правильной вложенности строки $w = abaabb$ выводится так.

$$\frac{}{S(\varepsilon)}(S \rightarrow \varepsilon) \quad \frac{S(\varepsilon)}{S(ab)}(S \rightarrow aSb) \quad \frac{S(ab)}{S(aabb)}(S \rightarrow aSb) \quad \frac{S(ab), S(aabb)}{S(abaabb)}(S \rightarrow SS)$$

Этот вывод можно записать в виде дерева доказательства, разделив $S(ab)$ для двух разных подстрок ab .

$$\frac{\frac{\frac{S(\varepsilon)}{S(ab)}}{S(aabb)} \quad \frac{S(\varepsilon)}{S(ab)}}{S(abaabb)}$$

Это тот же объект, что и дерево разбора на рис. 1.

Определение 1(D). Для грамматики $G = (\Sigma, N, R, S)$, высказывания имеют вид “строка w имеет свойство A ”, где $w \in \Sigma^*$ и $A \in N$, и обозначаются через $A(w)$.

Пусть $A \rightarrow u_0 B_1 u_1 \dots B_\ell u_\ell$ — правило, в котором $B_1, \dots, B_\ell \in N$, где $\ell \geq 0$ — это все нетерминальные символы, на которые оно ссылается, а $u_0, u_1, \dots, u_\ell \in \Sigma^*$ — символы между ними. Это правило позволяет сделать следующий логический вывод, для любых строк $v_1, \dots, v_\ell$, где над чертой — посылки, а под чертой — следствие.

$$\frac{B_1(v_1), \dots, B_\ell(v_\ell)}{A(u_0 v_1 u_1 \dots v_\ell u_\ell)}(A \rightarrow u_0 B_1 u_1 \dots B_\ell u_\ell) \quad (\text{for all } v_1, \dots, v_\ell \in \Sigma^*)$$

Вывод высказывания $A(u)$ — это последовательность таких шагов вывода, где в качестве посылок на каждом шаге используются ранее выведенные высказывания: $I_j \subseteq \{A_i(u_i) \mid i \in \{1, \dots, j-1\}\}$, для всех j .

$$\frac{I_1}{A_1(u_1)}, \quad \frac{I_2}{A_2(u_2)}, \quad \dots \quad \frac{I_{z-1}}{A_{z-1}(u_{z-1})}, \quad \frac{I_z}{A(u)}$$

Если такой вывод существует, это обозначается через $\vdash A(u)$.

Тогда, для всех $A \in N$, определяется $L_G(A) = \{w \mid \vdash A(w)\}$. Язык, задаваемый грамматикой — $L(G) = L_G(S) = \{w \mid \vdash S(w)\}$.

2.3 Определение через перезапись строк

Перезапись *сентенциальных форм* — строк над алфавитом $\Sigma \cup N$. Каждая такая строка — это схема предложения, в которой каждое вхождение каждого нетерминального символа $A \in N$ означает некоторую неуказанную строку со свойством A .

Перезапись начинается с S , что означает *любое правильное предложение*. На каждом шаге перезаписи некоторый нетерминальный символ A заменяется на правую часть правила для A , так что получается более определённая сентенциальная форма. Перезапись продолжается, пока не останутся только символы из Σ — то есть, предложение языка.

Пример 1(R). Для грамматики из примера 1, строка $abaabb$ получается перезаписью так.

$$S \Rightarrow SS \Rightarrow aSbS \Rightarrow abS \Rightarrow abaSb \Rightarrow abaaSbb \Rightarrow abaabb$$

Определение 1(R) (Хомский). Пусть $G = (\Sigma, N, R, S)$ — грамматика. Если $\eta A \theta$ — строка над совмещённым алфавитом символов и нетерминальных символов $\Sigma \cup N$ (где $A \in N$ и $\eta, \theta \in (\Sigma \cup N)^*$), и если $A \rightarrow \alpha$ — некоторое правило для A , тогда строка $\eta A \theta$ может быть **перезаписана в $\eta \alpha \theta$ за один шаг**, что обозначается следующим образом.

$$\eta A \theta \Rightarrow \eta \alpha \theta \quad (\text{для всех } A \rightarrow \alpha \in R \text{ and } \eta, \theta \in (\Sigma \cup N)^*)$$

Последовательность строк $\alpha_0, \alpha_1, \dots, \alpha_n$ над алфавитом $\Sigma \cup N$, где $n \geq 0$, называется *цепочкой перезаписи*, если всякая строка α_i может быть перезаписана за один шаг в строку α_{i+1} . Это обозначается так.

$$\alpha_0 \Rightarrow \alpha_1 \Rightarrow \dots \Rightarrow \alpha_n$$

Тогда говорится, что α_0 перезаписывается в α_n за n шагов (обозначение: $\alpha_0 \Rightarrow^n \alpha_n$). Также вводятся обозначения для перезаписи за ноль и более шагов ($\alpha \Rightarrow^* \beta$), за один и более шагов ($\alpha \Rightarrow^+ \beta$), и за не более чем n шагов ($\alpha \Rightarrow^{\leq n} \beta$).

Язык, задаваемый грамматикой.

$$\begin{aligned} L_G(A) &= \{w \mid w \in \Sigma^*, A \Rightarrow^+ w\} \\ L(G) &= L_G(S) \end{aligned} \quad (A \in N)$$

2.4 Определение через языковые уравнения

Гинзбург и Райс [1962]: грамматика представляется в виде системы уравнений, где языки — неизвестные. Всякий нетерминальный символ в грамматике $G = (\Sigma, N, R, S)$ становится *переменной*, принимающей значение языка над Σ . Значение переменной $A \in N$ в решении системы — множество строк, имеющих свойство A .



Рис. 2: Сеймур Гинзбург (1928–2004).

Эта *система языковых уравнений* содержит одно уравнение вида $A = \varphi_A$ для каждой переменной A , где φ_A — выражение с аргументами-языками, содержащее в себе все правила для нетерминального символа A . При преобразовании всякий оператор выбора переходит в операцию *объединения множеств*, конкатенация в правилах становится конкатенацией языков в уравнении, и всякий символ $a \in \Sigma$, использованный в правиле, становится одноэлементным языком $\{a\}$.

Пример 1(Е). Грамматика в примере 1 представляется следующим языковым уравнением, где объединение трёх конкатенаций в правой части соответствует трём правилам для S .

$$S = \underbrace{(\{a\} \cdot S \cdot \{b\})}_{S \rightarrow aSb} \cup \underbrace{(S \cdot S)}_{S \rightarrow SS} \cup \underbrace{\{\varepsilon\}}_{S \rightarrow \varepsilon}$$

φ_S

Одно из решений этого уравнения — язык Дика.

Уравнение можно прочитать так: “ w — правильно вложенная строка тогда и только тогда, когда она или имеет вид $w = aub$, где u — правильно вложенная, или представляется в виде конкатенации $w = uv$ двух правильно вложенных строк u, v , или это пустая строка $w = \varepsilon$ ”.

Это не единственное решение; есть, например, $S = \Sigma^*$. Язык, описываемый грамматикой, определяется с помощью *наименьшего решения* относительно частичного порядка поэлементного включения. Для данного уравнения, язык Дика — наименьшее решение.

Теорема 1 (Гинзбург и Райс [1962]). Пусть $X_1, \dots, X_n$ — переменные, и для каждой переменной X_i , пусть $\varphi_i(X_1, \dots, X_n)$ — выражение, содержащее переменные, любые постоянные языки, а также операции объединения и конкатенации. Тогда система n уравнений $X_i = \varphi_i(X_1, \dots, X_n)$, где $i \in \{1, \dots, n\}$, имеет решения, и среди них есть **наименьшее решение** $(L_1, \dots, L_n)$, то есть, такое, что всякое решение $(L'_1, \dots, L'_n)$ должно удовлетворять $L_i \subseteq L'_i$ для всех i .

Определение 1(Е) (Гинзбург и Райс [1962]). Пусть $G = (\Sigma, N, R, S)$ — грамматика. Соответствующая система языковых уравнений такова:

$$A = \bigcup_{\underbrace{A \rightarrow u_0 B_1 u_1 \dots B_\ell u_\ell \in R}_{\varphi_A}} \{u_0\} \cdot B_1 \cdot \{u_1\} \cdot \dots \cdot B_\ell \cdot \{u_\ell\} \quad (\text{for all } A \in N)$$

Пусть наименьшее решение имеет вид $A = L_A$, для всех $A \in N$. Тогда $L(G)$ определяется как L_S .

Пример 2. For the system of equations in Example 1(E), the sequence of approximations takes the following form. The initial approximation is the empty set.

$$L^{(0)} = \emptyset$$

At the first step, the empty set is instantiated into the right-hand side of the equation: $(\{a\} \cdot \emptyset \cdot \{b\}) \cup (\emptyset \cdot \emptyset) \cup \{\varepsilon\} = \{\varepsilon\}$. That is, nothing comes out from the first two sets yet, whereas the third set provides the empty string.

$$L^{(1)} = \{\varepsilon\}$$

At the second step, the language instantiated into the right-hand side is now $\{\varepsilon\}$, and a pair of brackets get wrapped around the empty string according to the first rule, resulting in the string ab . Altogether the right-hand side equals $(\{a\} \cdot \{\varepsilon\} \cdot \{b\}) \cup (\{\varepsilon\} \cdot \{\varepsilon\}) \cup \{\varepsilon\} = \{\varepsilon, ab\}$.

$$L^{(2)} = \{\varepsilon, ab\}$$

Under the subsequent substitution of the language $\{\varepsilon, ab\}$ into the right-hand side, first, a pair of brackets gets wrapped around the string ab as well, and second, the concatenation of ab with ab appears for the first time. Thus, the third approximation is $(\{a\} \cdot \{\varepsilon, ab\} \cdot \{b\}) \cup (\{\varepsilon, ab\} \cdot \{\varepsilon, ab\}) \cup \{\varepsilon\} = \{\varepsilon, ab, aabb, abab\}$.

$$L^{(3)} = \{\varepsilon, ab, aabb, abab\}$$

Proceeding in this way, all well-nested strings are gradually obtained.

$$\begin{aligned} L^{(4)} &= \{\varepsilon, ab, aabb, abab, aaabbb, aababb, abaabb, ababab, aabbab, aabbaabb, aabbabab, ababaabb, abababab\} \\ &\vdots \end{aligned}$$

The purpose of this section is to establish the correctness of Definition 1(E) by proving that every system of language equations from a certain class always has a least solution. The proof shall be given in a sufficiently general form, so that later in this book it could be reused for several more general kinds of grammars.

Let Σ be an alphabet, let $n \geq 1$. Consider vectors on n languages of the form $(L_1, \dots, L_n)$; the set of all such vectors is $(2^{\Sigma^*})^n$. Let these vectors be partially ordered with respect to componentwise inclusion, so that a vector $(K_1, \dots, K_n)$ is deemed to be less than or equal to

another vector $(L_1, \dots, L_n)$, denoted by $(K_1, \dots, K_n) \sqsubseteq (L_1, \dots, L_n)$, if $K_i \subseteq L_i$ for every i . The least element with respect to this is $\perp = (\emptyset, \dots, \emptyset)$, the greatest one is $\top = (\Sigma^*, \dots, \Sigma^*)$. For any two such vectors, their componentwise union is denoted by $(K_1, \dots, K_n) \sqcup (L_1, \dots, L_n) = (K_1 \cup L_1, \dots, K_n \cup L_n)$.

Let $\varphi = (\varphi_1, \dots, \varphi_n)$ be a vector function representing the right-hand side of the system. It is assumed that φ has the following two properties.

- First, φ is *monotone*, in the sense that for any two vectors K and L , the inequality $K \sqsubseteq L$ implies $\varphi(K) \sqsubseteq \varphi(L)$.
- Second, φ is $\sqcup$ -*continuous*, in the sense that for every increasing sequence of vectors of languages $\{L^{(i)}\}_{i=1}^\infty$, applying φ to its union produces the same value as applying φ to each element and taking the union.

$$\bigsqcup_{i=1}^\infty \varphi(L^{(i)}) = \varphi\left(\bigsqcup_{i=1}^\infty L^{(i)}\right).$$

Лемма 1. *A function on languages φ formed from variables and constant languages using concatenation and union is monotone and continuous.*

Доказательство. By induction on the structure of individual expressions. Variables and constants are monotone, union/concatenation of monotone functions is monotone. Same with continuity. $\square$

Лемма 2 (Клини). *Если оператор φ монотонен и $\sqcup$ -непрерывен, то у системы уравнений $X = \varphi(X)$ существует наименьшее решение — следующий вектор.*

$$L = \bigsqcup_{k=0}^\infty \varphi^k(\perp)$$

Доказательство. Это классическое доказательство, работающее для любого частично упорядоченного множества с наименьшим элементом, и для любой монотонной и $\sqcup$ -непрерывной функции φ .

Сперва показывается, что последовательность $\{\varphi^k(\perp)\}_{k \geq 0}$ монотонна.

$$\perp \sqsubseteq \varphi(\perp) \sqsubseteq \varphi^2(\perp) \sqsubseteq \dots \sqsubseteq \varphi^k(\perp) \sqsubseteq \dots$$

Действительно, $\perp \sqsubseteq \varphi(\perp)$ по определению наименьшего элемента, и далее из каждого неравенства $\varphi^{k-1}(\perp) \sqsubseteq \varphi^k(\perp)$ по монотонности оператора φ вытекает следующее неравенство $\varphi^k(\perp) \sqsubseteq \varphi^{k+1}(\perp)$.

To see that the infinite union of approximations $L = \bigsqcup_{k=0}^\infty \varphi^k(\perp)$ is the solution of the system, consider the value of φ on L . Since φ is $\sqcup$ -continuous, this value can be expressed as follows.

$$\varphi(L) = \varphi\left(\bigsqcup_{k=0}^\infty \varphi^k(\perp)\right) = \bigsqcup_{k=0}^\infty \varphi(\varphi^k(\perp)) = \bigsqcup_{k=0}^\infty \varphi^{k+1}(\perp)$$

Thus, $\varphi(L)$ is the limit of the following sequence.

$$\varphi(\perp) \sqsubseteq \varphi^2(\perp) \sqsubseteq \varphi^3(\perp) \sqsubseteq \dots \sqsubseteq \varphi^{k+1}(\perp) \sqsubseteq \dots$$

Since this is the same sequence as the sequence converging to L , their infinite unions must be the same, that is, $L = \varphi(L)$. This proves that L is a solution.

The last claim is that L is the least among all solutions. Let K be any vector with $\varphi(K) = K$. For the two sequences $\{\varphi^k(\perp)\}_{k=0}^\infty$ and $\{\varphi^k(K)\}_{k=0}^\infty$, each element of the former sequence is a subset of the corresponding element of the latter sequence: this is established inductively on k , using the monotonicity of φ .

$$\begin{array}{ccccccccccc} \perp & \sqsubseteq & \varphi(\perp) & \sqsubseteq & \varphi^2(\perp) & \sqsubseteq & \dots & \sqsubseteq & \varphi^k(\perp) & \sqsubseteq & \dots \rightarrow L \\ \sqcap & & \sqcap & & \sqcap & & & & \sqcap & & \\ K & \sqsubseteq & \varphi(K) & \sqsubseteq & \varphi^2(K) & \sqsubseteq & \dots & \sqsubseteq & \varphi^k(K) & \sqsubseteq & \dots \end{array}$$

These inequalities between the corresponding elements is extended to the least upper bounds of the sequences: L is less than or equal to the limit of the sequence $\{\varphi^k(K)\}_{k=0}^\infty$. However, every element of the latter sequence is K , because $\varphi(K) = K$. This proves that $L \sqsubseteq K$, and confirms that L is the least among all solutions. $\square$

2.5 Равносильность четырёх определений

Все четыре определения равносильны, то есть они задают те же значения $L_G(A)$ и $L(G)$.

Теорема 2. Пусть $G = (\Sigma, N, R, S)$ — грамматика, как в определении 1. Для всякого нетерминального символа $A \in N$ и для всякой строки $w \in \Sigma^*$, следующие утверждения равносильны:

- (T). существует дерево разбора w из A ;
- (D). высказывание $A(w)$ выводимо ($\vdash A(w)$);
- (R). A можно переписать в w за один и более шагов ($A \Rightarrow^* w$);
- (E). w принадлежит A -компоненту наименьшего решения системы языковых уравнений ($w \in L_A$).

Доказательство. The proof is by a circle of four implications $((T) \Rightarrow (D) \Rightarrow (R) \Rightarrow (E) \Rightarrow (T))$.

Each implication is proved by induction on some measure of derivation complexity: for parse trees, that is *the height of the tree* (that is, the length of the longest path from the root to a leaf); for logical derivations, this is the number of steps in the derivation; for string rewriting, this is the number of rewriting steps; and for language equations, this is the number of iterations until the string appears in the current approximation.

In each part, it would be possible to identify a base case, but no need to do that.

$\boxed{(T) \Rightarrow (D)}$ Induction on the height of the trees.

Assume that there is a parse tree of w from A . Its root is labelled with A . Let the immediate successors of the root be labelled with the symbols $u_0 B_1 u_1 \dots B_\ell u_\ell$, where $B_1, \dots, B_\ell \in N$, are all nonterminal symbols among these successors, and $u_0, \dots, u_\ell$ are symbols of the alphabet. Then, the rule applied at the root node is $A \rightarrow u_0 B_1 u_1 \dots B_\ell u_\ell$, and the tree is of the form shown in Figure 3. In particular, for each B_i , there is a subtree of some substring from B_i . Let v_i be that substring. The whole string w is therefore of the form $w = u_0 v_1 u_1 \dots v_\ell u_\ell$.

Since the height of each subtree of v_i from B_i is less than the height of the whole tree for A , the induction hypothesis asserts that the proposition $B_i(v_i)$ is derivable. Then, the desired

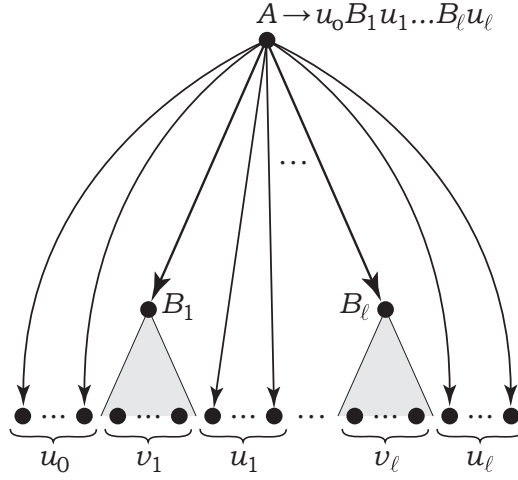


Рис. 3: Parse tree of a string $w = u_0 v_1 u_1 \dots v_\ell u_\ell$ from A .

proposition $A(w)$ is derived in one extra step.

$$\frac{B_1(v_1) \dots B_\ell(v_\ell)}{A(u_0 v_1 u_1 \dots v_\ell u_\ell)} (A \rightarrow u_0 B_1 u_1 \dots B_\ell u_\ell)$$

$\boxed{(D) \Rightarrow (R)}$ Induction on the number of steps in the derivation of $A(w)$.

Consider the last step of the derivation of $A(w)$, which must be of the following form, for some rule $A \rightarrow u_0 B_1 u_1 B_2 \dots u_{\ell-1} B_\ell u_\ell$, and for some strings $v_1, \dots, v_\ell \in \Sigma^*$.

$$\frac{B_1(v_1) \dots B_\ell(v_\ell)}{A(w)} (A \rightarrow u_0 B_1 u_1 \dots B_\ell u_\ell)$$

Then, w is $u_0 v_1 u_1 \dots v_\ell u_\ell$, and each of the premises $B_i(v_i)$ has a shorter derivation than $A(w)$.

Then, by the induction hypothesis, each B_i can be rewritten to v_i in zero or more steps ($B_i \Rightarrow^* v_i$). Using these rewriting sequences, the following rewriting sequence from A to w can be constructed.

$$A \Rightarrow u_0 B_1 u_1 \dots B_\ell u_\ell \Rightarrow^* u_0 v_1 u_1 \dots v_\ell u_\ell$$

$\boxed{(R) \Rightarrow (E)}$ Induction on the number of steps needed to rewrite A to w .

Assume that A can be rewritten to w , and let $A \rightarrow u_0 B_1 u_1 B_2 \dots u_{\ell-1} B_\ell u_\ell$ be the rule used at the first step of that rewriting sequence. Then, this sequence further proceeds by rewriting each nonterminal symbol B_i to some substring v_i , where w is of the form $u_0 v_1 u_1 \dots v_\ell u_\ell$.

$$A \Rightarrow u_0 B_1 u_1 \dots B_\ell u_\ell \Rightarrow^* u_0 v_1 u_1 \dots v_\ell u_\ell$$

For each B_i , the rewriting sequence from B_i to v_i is shorter than the one from A to w , and therefore, by the induction hypothesis, v_i is in the component L_{B_i} of the least solution.

By Lemma 2, this component is an infinite union of $L_{B_i}^{(k)}$ over all k . Then there exists such a number $k_i \geq 1$, that v_i belongs to $L_{B_i}^{(k_i)}$. Furthermore, by the monotonicity of the sequence of approximations, for the step $k = \max\{k_1, \dots, k_\ell\}$, each v_i is in corresponding $L_{B_i}^{(k)}$. Then, w is in $\varphi_A(L_1^{(k)}, \dots, L_n^{(k)})$. Since the latter is exactly $L_A^{(k+1)}$, the next approximation of A the string w is therefore in L_A .

$\boxed{(E) \Rightarrow (T)}$ If w belongs to the A -component of the least solution, then, at some iteration, w appears in the approximation of A . The proof is by induction on the number of iterations until w appears there.



Рис. 4: Ноам Хомский (род. 1928).

Assume that $w \in L_A^{(k+1)}$, where $k \geq 0$. Since $L_A^{(k+1)}$ is defined as the value of φ_A on the approximations of all variables at the previous step, $L_A^{(k+1)} = \varphi_A(L_1^{(k)}, \dots, L_n^{(k)})$, there exists a rule $A \rightarrow u_0 B_1 u_1 B_2 \dots u_{\ell-1} B_\ell u_\ell$ for which w belongs to the language $\{u_0\} \cdot L_{B_1}^{(k)} \cdot \{u_1\} \cdot \dots \cdot L_{B_\ell}^{(k)} \cdot \{u_\ell\}$. This in turn means that there is a partition of w as $w = u_0 v_1 u_1 \dots v_\ell u_\ell$, where each v_i is in $L_{B_i}^{(k)}$. By the induction hypothesis for each v_i , there is a parse tree of v_i from B_i . Then these parse trees can be combined into a larger tree of w from A , as shown in Figure 3. $\square$

3 Нормальные виды грамматик

3.1 Пустые правила

(было на 2-м курсе, надо вспомнить) Выселение символов алфавита в отдельные правила. Нарезка длинных правил. Удаление пустых правил. Тогда рост грамматики не более чем утраивается. Поэтому линейный рост.

3.2 Единичные правила

(было на 2-м курсе, надо вспомнить) Удаление единичных правил. Рост: квадратичный. Нижняя оценка около $n^{3/2}$ получена Блумом [1983].

3.3 Нормальный вид Хомского

(было на 2-м курсе, надо вспомнить) Обыкновенная грамматика $G = (\Sigma, N, R, S)$ находится в нормальном виде Хомского, если всякое правило из R имеет следующий вид.

$$\begin{aligned} A &\rightarrow BC && (B, C \in N) \\ A &\rightarrow a && (a \in \Sigma) \\ S &\rightarrow \varepsilon && (\text{только если } S \text{ не используется в правых частях никаких правил}) \end{aligned}$$

Теорема 3 (Хомский [1959]). *Для всякой обыкновенной грамматики можно построить грамматику в н.в. Хомского, задающую тот же язык.*

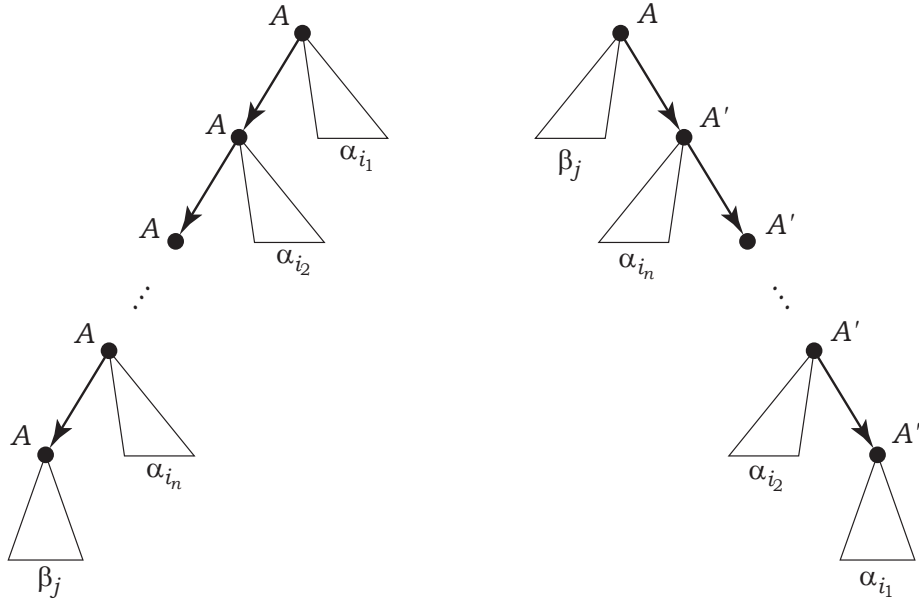


Рис. 5: Eliminating direct left recursion: (слева) parse tree in the original grammar; (справа) parse tree in the transformed grammar.

3.4 Левая рекурсия

Предложения обычно пишут символ за символом, слева направо, и большинство человеческих языков предназначены для последовательной односторонней обработки символов. При таком анализе, всякий раз, когда наблюдается начало некоторой подстроки, нужно решить — возможно, недетерминированно, — какое правило грамматики описывает эту подстроку, до того, как вся эта подстрока будет прочитана до конца. С этой точки зрения, правила вида $A \rightarrow A\alpha$ в высшей степени неудобны, поскольку они ничего не говорят о строении начала текущей подстроки. Такие правила называются *непосредственно лево-рекурсивными*.

Непосредственная левая рекурсия в правилах для нетерминального символа A показана на рис. 5(left). Здесь самый левый путь в дереве разбора использует произвольное число леворекурсивных правил вида $A \rightarrow A\alpha$, пока левая рекурсия наконец не завершается правилом $A \rightarrow \beta$, где β начинается не с A . Так леворекурсивный нетерминальный символ определяет строки общего вида $\beta\alpha_1 \dots \alpha_n$, перечисляя $\alpha_n, \dots, \alpha_1$ справа налево. In order to eliminate the left recursion, the same constituents should be defined in the reverse order: first, β is appended at the left at the top of the parse tree, and then, all $\alpha_1, \dots, \alpha_n$ are listed from left to right. The desired parse tree is presented in Figure 5(right), and the next lemma shows how this can be achieved in an arbitrary grammar.

Лемма 3 (Elimination of direct left recursion). Пусть $G = (\Sigma, N, R, S)$ — обыкновенная грамматика и пусть для нетерминального символа A есть m правил с непосредственной левой рекурсией и n без неё.

$$A \rightarrow A\alpha_1 \mid \dots \mid A\alpha_m \mid \beta_1 \mid \dots \mid \beta_n$$

Здесь ни одна из конкатенаций $\beta_1, \dots, \beta_n$ не начинается с A .

Тогда эти правила можно заменить на следующие правила, где A' — новый нетерминальный символ, так что полученная грамматика $G' = (\Sigma, N \cup \{A'\}, R', S)$ задаёт тот

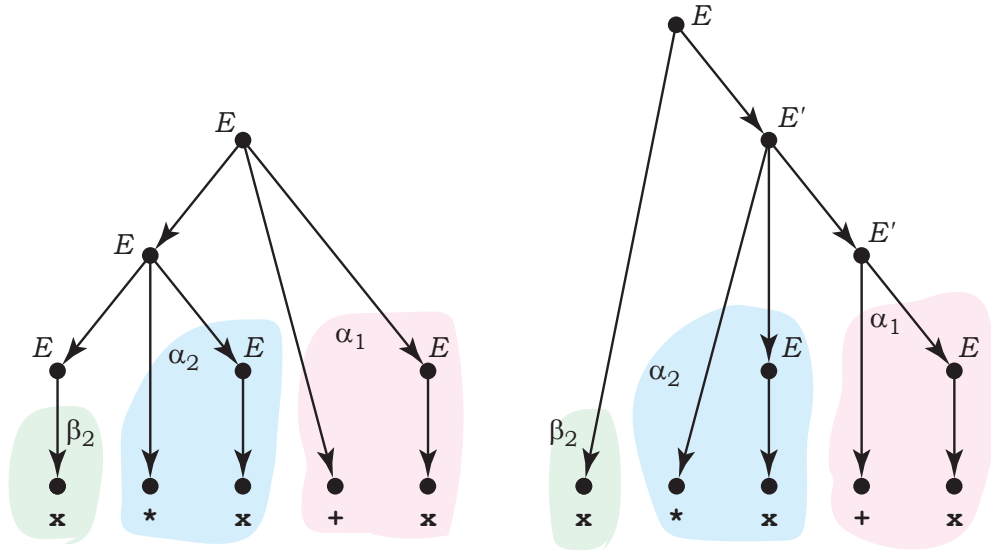


Рис. 6: Eliminating direct left recursion in Example 3: (слева) parse tree in the original grammar; (справа) parse tree in the transformed grammar.

исе язык.

$$A \rightarrow \beta_1 \mid \dots \mid \beta_n \mid \beta_1 A' \mid \dots \mid \beta_n A'$$

$$A' \rightarrow \alpha_1 A' \mid \dots \mid \alpha_m A' \mid \alpha_1 \mid \dots \mid \alpha_m$$

Пример 3. The following grammar for arithmetical expressions has left recursion in the first two rules.

$$E \rightarrow \underbrace{E + E}_{\alpha_1} \mid \underbrace{E * E}_{\alpha_2} \mid \underbrace{(E)}_{\beta_1} \mid \underbrace{1}_{\beta_2}$$

According to Lemma 3, it is equivalently transformed to the grammar below.

$$E \rightarrow (E) \mid 1 \mid (E)E' \mid 1E'$$

$$E' \rightarrow +EE' \mid *EE' \mid +E \mid *E$$

Parse trees of the string $xx*x+x$ according to both grammars are given in Figure 6.

3.5 Нормальный вид Грейбах

Непосредственная левая рекурсия — это не единственное препятствие обработке строки слева направо. Существует более общее понятие *левой зависимости*, проявляющейся в правилах вида $A \rightarrow B\alpha$. Цепочка левых зависимостей может свернуться в *косвенную левую рекурсию* в правилах вида $A_1 \rightarrow A_2\alpha_1$, $A_2 \rightarrow A_3\alpha_2$, $\dots$, $A_n \rightarrow A_1\alpha_n$.

Следующий нормальный вид запрещает любые левые зависимости.



Рис. 7: Шейла Грейбах (род. 1939), Дэниел Розенкранц (род. 1943), Фридрих Урбанек.

Определение 2 (Грейбах [1965]). Обыкновенная грамматика $G = (\Sigma, N, R, S)$ находится в нормальном виде Грейбах, если всякое правило из R имеет один из двух следующих видов.

$$\begin{aligned} A &\rightarrow a\alpha & (a \in \Sigma, \alpha \in (\Sigma \cup N)^*) \\ S &\rightarrow \varepsilon & (\text{только если } S \text{ не встречается в правых частях никаких правил}) \end{aligned}$$

Грамматика G находится в k -нормальном виде Грейбах, если, кроме того, $|\alpha| \leq k$ в каждом правиле первого вида.

Теорема 4 (Грейбах [1965], с улучшениями Урбанека [1985] и Блума и Коха [1999]). Для всякой обыкновенной грамматики существует и может быть эффективно построена грамматика в 2-нормальном виде Грейбах, задающая тот же язык. Размер полученной грамматики — $O(n^4)$ относительно размера исходной, и $O(n^3)$, если исходная грамматика находится в нормальном виде Хомского.

Первоначальное построение, предложенное самой Грейбах [1965], приводило к экспоненциальному росту размера грамматики. Полиномиальное преобразование было впервые получено Розенкранцем [1967]. Предлагаемое ниже построение принадлежит Урбанеку [1985], и оно обобщает метод удаления левой рекурсии, представленный в лемме 3.

Пусть G — грамматика в нормальном виде Хомского. Если дерево разбора некоторой строки начинается с использования правил с левыми зависимостями, как показано на рис. 8(левом), то пусть $A \rightarrow B_1 C_1, B_1 \rightarrow B_2 C_2, \dots, B_{n-1} \rightarrow B_n C_n$ — цепочка этих правил, которая рано или поздно заканчивается символом, по некоторому правилу $B_n \rightarrow a$. Эта цепочка правил по существу задаёт конкатенацию $aC_n \dots C_2 C_1$, и цель — определить ту же самую конкатенацию, не используя никаких левых зависимостей.

В новой грамматике эта цепочка воспроизводится в обратном порядке, как показано на рис. 8(правом). Символ a задаётся в самом начале цепочки, в правиле $A \rightarrow a B_n \setminus A$, где нетерминальный символ $B_n \setminus A$ означает «строка вида A , с удалённым префиксом вида B_n ». Этот нетерминальный символ определяется правилом $B_n \setminus A \rightarrow C_n B_{n-1} \setminus A$, которое воспроизводит правило $B_{n-1} \rightarrow B_n C_n$ исходной грамматики. Обратная цепочка заканчивается правилом $B_1 \setminus A \rightarrow C_1 A \setminus A$, где $A \setminus A$ означает «строка вида A , с удалённым префиксом вида A », и соответственно имеет правило $A \setminus A \rightarrow \varepsilon$, соответствующее случаю, когда удалённый префикс — это вся строка. В целом, новая цепочка определяет ту же самую конкатенацию

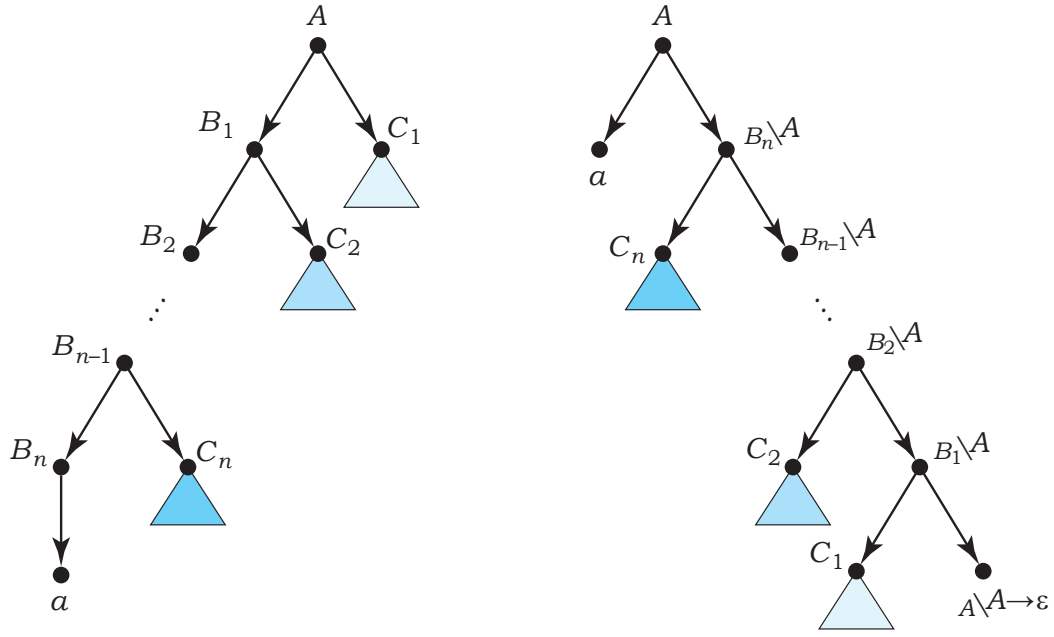


Рис. 8: Преобразование к нормальному виду Грейбах в лемме 4: (слева) дерево разбора в исходной грамматике; (справа) соответствующее дерево в преобразованной грамматике.

$aC_n \dots C_2C_1$, что и в исходной грамматике, что нетрудно видеть, сравнив два дерева на рис. 8.

Следующая лемма определяет это преобразование для произвольной грамматики, используя нетерминальные символы вида $B \setminus A$, определённые для всякой пары нетерминальных символов A, B в исходной грамматике. Всякий нетерминальный символ $B \setminus A$ означает «любая строка вида A , с удалённым префиксом вида B ». Точнее, он должен задавать множество всех таких строк w , что существует дерево разбора с корнем A и с дыркой вместо B -поддерева, причём справа от удалённого поддерева находятся листья w , а слева никаких листьев нет.

Лемма 4 (Урбанек [1985]). Пусть $G = (\Sigma, N, R, S)$ — обыкновенная грамматика в нормальном виде Хомского. Тогда грамматика $G' = (\Sigma, N \cup N', R', S)$, где $N' = \{B \setminus A \mid A, B \in N\}$, а R' содержит приведённые ниже правила, задаёт тот же язык.

$$\begin{array}{ll}
 D \rightarrow a_A D & (D \in N, A \rightarrow a \in R) \\
 B \setminus D \rightarrow C_A \setminus D & (D \in N, A \rightarrow BC \in R) \\
 D \setminus D \rightarrow \epsilon & (D \in N)
 \end{array}$$

В частности, $L_{G'}(A) = L_G(A)$ для всех $A \in N$, а всякий нетерминальный символ $A \setminus D$, где $D, A \in N$, задаёт множество всех строк w , для которых Aw — это D -сентенциальная форма.

$$L_{G'}(A \setminus D) = \{w \mid \text{есть дерево } Aw \text{ из } D, \text{ где } A \text{ — дырка, справа от которой — листья } w\}$$

(доказательство будет на следующей лекции)

Список литературы

- [1983] N. Blum, “More on the power of chain rules in context-free grammars”, *Theoretical Computer Science*, 27 (1983), 287–295.
- [1999] N. Blum, R. Koch, “Greibach normal form transformation revisited”, *Information and Computation*, 150:1 (1999), 112–118.
- [1959] N. Chomsky, “On certain formal properties of grammars”, *Information and Control*, 2:2 (1959), 137–167.
- [1967] S. Ginsburg, S. A. Greibach, M. A. Harrison, “One-way stack automata”, *Journal of the ACM*, 14:2 (1967), 389–418.
- [1962] S. Ginsburg, H. G. Rice, “Two families of languages related to ALGOL”, *Journal of the ACM*, 9 (1962), 350–371.
- [1963] S. Ginsburg, G. Rose, “Operations which preserve definability in languages”, *Journal of the ACM*, 10:2 (1963), 175–195.
- [1965] S. A. Greibach, “A new normal-form theorem for context-free phrase structure grammars”, *Journal of the ACM*, 12 (1965), 42–52.
- [1967] D. J. Rosenkrantz, “Matrix equations and normal forms for context-free grammars”, *Journal of the ACM*, 14:3 (1967), 501–507.
- [1985] F. J. Urbanek, “On Greibach normal form construction”, *Theoretical Computer Science*, 40 (1985), 315–317.